

# Assessment Paper 1: Software Development

## Section A: Technical Questions

**Q1. What is the difference between a compiler and an interpreter? Give one example of each.**

### Compiler:

A compiler converts the whole program into machine code before running it.

After compilation, it creates an executable file that can run without the compiler.

Eg- C, C++

### Interpreter:

An interpreter converts and executes the program one line at a time while it is running.

The interpreter must be available every time the program is executed.

Eg- Python

**Q2. Explain the difference between an array and a linked list. When would you prefer one over the other?**

### Array:

- ✚ An array stores elements in continuous memory locations.
- ✚ It allows quick access to elements using their index, but adding or removing elements can be difficult because other elements may need to be shifted.
- ✚ Use an array when you need fast access to data by index.

### Linked List:

- ✚ A linked list stores elements as separate nodes connected by pointers.
- ✚ It is easy to add or remove elements, but accessing a specific element takes more time because the list must be traversed from the beginning.
- ✚ Use a linked list when you frequently add or remove elements and the size of the data may change.

**Q3. What is Object-Oriented Programming? Briefly explain any two OOP principles with examples.**

Object-Oriented Programming is a programming approach that organizes code using objects and classes. It helps make programs more structured, reusable, and easier to maintain.

**Encapsulation:**

Encapsulation means combining data and methods into a single unit (class) and restricting direct access to some data.

Eg- A bank account class keeps the account balance private and allows access only through methods like deposit and withdraw.

**Inheritance:**

Inheritance allows one class to acquire the properties and methods of another class, promoting code reuse.

Eg- A Car class can inherit common features such as speed and fuel from a Vehicle class.

**Q4. What is the difference between SQL and NoSQL databases? Give one example of each.****SQL:**

- + Stands for Structured Query Language
- + Relational Database Management System (RDBMS)
- + Suitable for structured data with predefined schema
- + Data are stored in tables with rows and columns
- + Supports Joins and complex Queries
- + Uses normalized data structures
- + Eg- MySQL, PostgreSQL

**NoSQL:**

- + Stands for Not Only SQL
- + Non-Relational Database Management System
- + Suitable for Unstructured and semi-structured data
- + Data stored in collections or documents
- + Does not support Joins or Complex Queries
- + Uses denormalized data structures
- + Eg-MongoDB

**Q5. Explain what an API is and why it is useful in software development.**

An API, or Application Programming Interface, is a set of rules and protocols that allows different software applications to communicate and share data with each other.

- It enables different systems to work together easily.
- It allows developers to use existing services instead of building everything from scratch.
- It saves development time and effort.
- It helps in integrating third-party services such as payment gateways, maps, and social media platforms.

**Q6. What is version control? Name a popular version control tool and explain one basic command you would use.**

A Version Control System (VCS) is a tool used in software development and collaborative projects to track and manage changes to source code.

Tracks and records changes to the codebase, maintaining a structured project history.

Allows multiple developers to collaborate on the same project without overwriting each other's work.

Eg- Git

- `git add .`
- `git commit -m "First Commit"`
- `git push origin main`

**Q7. What is the difference between frontend and backend development?**

**Frontend Development:**

- ✚ Frontend development focuses on the part of a website or application that users can see and interact with.
- ✚ It involves designing and building the user interface using technologies like HTML, CSS, and JavaScript.
- ✚ Frontend deals with the user interface and user experience.

**Backend Development:**

- ✚ Backend development focuses on the server-side functionality of an application.
- ✚ It handles business logic, databases, authentication, and data processing using technologies like Python, Django, Java, or Node.js.
- ✚ Backend deals with data processing, server operations, and database management.

**Q8. Explain the concept of time complexity with a simple example (e.g., comparing a loop vs nested loop).**

- Time complexity is a way to measure how the execution time of a program increases as the input size grows.
- It helps us understand the efficiency of an algorithm.
- A single loop is generally faster and more efficient than a nested loop when working with large amounts of data.
- Time complexity helps developers choose better algorithms and improve application performance.
- A single loop that runs through  $n$  elements has a time complexity of  $O(n)$  because the number of operations increases linearly with the input size.

- A nested loop where one loop runs inside another for  $n$  elements has a time complexity of  $O(n^2)$  because the total number of operations grows much faster as the input size increases.

**Q9. What is debugging? Describe your general approach when you encounter an error in your code.**

Debugging is the process of finding, analyzing, and fixing errors or bugs in a program to make it work correctly.

**Approaches:**

- ✚ Read the error message carefully to understand the issue.
- ✚ Identify the part of the code where the error occurred.
- ✚ Check the logic, syntax, and variable values.
- ✚ Use print statements or a debugger to trace the program's execution.
- ✚ Fix the issue and test the code again to ensure it works as expected.

**Q10. What is the difference between a stack and a queue? Give a real-world example of each.**

**Stack:**

- ✚ A stack is a data structure that follows the Last In, First Out (LIFO) method.
- ✚ The last element added is the first one to be removed.
- ✚ Eg: A stack of plates, the last plate placed on top is the first plate taken off.

**Queue:**

- ✚ A queue is a data structure that follows the First In, First Out (FIFO) principle.
- ✚ The first element added is the first one to be removed.
- ✚ Eg: A queue of people waiting at a ticket counter, the person who arrives first is served first.

## **Section B: Behavioral Questions**

**Q11. Describe a time when you had to learn a new technology or tool quickly on your own. How did you approach it?**

Recently, I had to learn Django REST Framework (DRF) quickly while building a backend project. I started by understanding the basics through documentation and online tutorials. Then, I practiced by creating small APIs and gradually integrated them into my project. Whenever I faced issues, I referred to the official documentation, searched for solutions online, and experimented with different approaches. By applying what I learned in a real project, I was able to understand the

framework more effectively and complete the required functionality. This experience improved my ability to learn new technologies independently and adapt to new challenges quickly.

**Q12. Tell us about a project (academic or personal) you are most proud of. What was your specific contribution?**

One project that I am most proud of is Vehicle Assist, a web-based platform designed to help users during vehicle breakdowns by connecting them with nearby service providers and assistance resources. My specific contribution was developing the backend using Python and Django. I designed the database models, implemented user authentication, created APIs and views for managing service requests, and handled the integration between different modules of the application. I also worked on the frontend using HTML, CSS, and JavaScript to create a user-friendly interface. Through this project, I gained hands-on experience in full-stack development, database management, and problem-solving while building a practical solution for real-world vehicle assistance needs.

**Q13. How do you usually handle situations where you are stuck on a problem for a long time? What do you do next?**

When I am stuck on a problem for a long time, I first break it down into smaller parts and try to identify the exact point where the issue is occurring. I carefully review the error messages, check my code logic, and use debugging techniques such as print statements or debugging tools. If I still cannot solve it, I refer to official documentation, online resources, and relevant examples to understand different approaches. I also try to reproduce the problem in a smaller code sample to isolate the issue. If needed, I will AI to find the error and fix it. Once I find a solution, I make sure to understand the root cause so that I can avoid similar issues in the future. This approach helps me stay patient, learn continuously, and solve problems effectively.

**Q14. Describe a situation where you had to work as part of a team. What role did you play?**

If I want to work as part of a team, my role was primarily focused on the backend development using Python and Django or FastAPI. I was responsible for designing database models, implementing application logic, and integrating different modules. I regularly communicated with my teammates to understand their requirements and ensure that the frontend and backend components worked together smoothly. Whenever issues arose, we discussed them as a team, shared ideas, and helped each other find solutions. Working in a team will help to improve both my technical skills and my ability to coordinate effectively with others.

**Q15. How do you prioritize tasks when you have multiple deadlines at the same time?**

When I have multiple deadlines, I first list all the tasks and understand their priority based on urgency, importance, and impact. I then break larger tasks into smaller steps and create a plan to

manage my time effectively. I focus on completing high-priority and time-sensitive tasks first while ensuring steady progress on the remaining work. I also track my progress regularly and adjust my schedule if priorities change. By staying organized and planning ahead, I am able to meet deadlines efficiently without compromising the quality of my work.

**Q16. Imagine you receive two job offers: Offer A has a higher salary but a very structured, narrow role with limited learning. Offer B has a modest salary but offers exposure to multiple technologies, mentorship, and the flexibility to work from home. Which would you lean towards, and why?**

As someone who is in the early stage of my career, I would lean towards Offer B. While salary is important, I believe gaining strong technical skills, working with different technologies, and learning from experienced mentors will have a greater impact on my long-term career growth. Exposure to multiple technologies would help me build a broader skill set, and mentorship would allow me to learn industry best practices more quickly. The flexibility to work from home is also an added benefit that can help maintain a good work-life balance. I would choose the opportunity that helps me learn, grow, and become a better developer, even if the initial salary is slightly lower.

**Q17. What kind of work environment helps you perform and learn best — explain with reasons.**

I perform and learn best in a collaborative and supportive work environment where team members are open to sharing knowledge and helping each other grow. As a developer, I enjoy working in a place where I can ask questions, receive constructive feedback, and learn from experienced colleagues. I also appreciate an environment that encourages continuous learning, problem-solving, and taking ownership of tasks. Having opportunities to work on real projects and explore new technologies helps me improve my technical skills and stay motivated. Such an environment allows me to contribute effectively while continuously developing both my technical and professional abilities.

**Q18. Describe a mistake you made while working on a project and what you learned from it.**

While working on my Vehicle Assist project, I made a mistake in the database design during the initial stages. I didn't properly plan the relationships between some models, which led to issues when adding new features later. After identifying the problem, I reviewed the requirements, redesigned the models, updated the database structure, and thoroughly tested the application to ensure everything worked correctly. From this experience, I learned the importance of spending enough time on planning and designing before starting development. It also taught me to test changes carefully and think about future scalability when building applications.

**Q19. If you joined a small, growing team where you might need to wear multiple hats (coding, testing, learning new tools) instead of a single fixed role, how would you feel about that, and why?**

I would be excited about that opportunity. As someone who is still building my career, working in a small and growing team would allow me to gain experience in different areas such as development, testing, debugging, and learning new tools. I enjoy learning new technologies and taking on challenges that help me improve my skills. Being involved in multiple aspects of a project would give me a better understanding of the software development process and help me become a more versatile developer. I believe this kind of environment would help me grow faster, contribute more effectively to the team, and develop both my technical and problem-solving abilities.

**Q20. Where do you see yourself professionally in the next 2-3 years, and what kind of opportunities matter most to you in getting there?**

In the next 2–3 years, I see myself growing into a skilled Full-Stack Developer, with strong expertise in Python, Django, REST APIs, databases, and modern frontend technologies. I want to work on real-world projects, contribute to meaningful products, and continuously improve my technical and problem-solving skills. The opportunities that matter most to me are those that provide continuous learning, hands-on experience, mentorship, and exposure to different technologies. I value an environment where I can take on new challenges, collaborate with experienced professionals, and contribute to impactful projects. My goal is to build a strong foundation as a software developer and become someone who can take ownership of projects and deliver reliable solutions.